1 Wprowadzenie

To, co dzisiaj jest nam niezbędne, pokażemy sobie na tablicy.

2 Zadania

2.1 Zadanie 0

Proszę (już tradycyjnie) uporządkować swoje repozytorium, aby organizacja projektów wyglądała jak poniżej. Oczywiście w tej strukturze mogą znajdować się również inne katalogi i pliki, ale proszę dokładnie odwzorować przedstawiony porządek projektów.

```
.korzeń_repozytorium
+-- CW_0
|   +-- tak, jak poprzednio
|
+-- CW_1
|   +-- AiSD2026ZEx1
|       +-- pom.xml
|
+-- CW_2
|   +-- AiSD2026ZEx2
|       +-- pom.xml
|
+-- CW_3
|   +-- AiSD2026ZEx3
|       +-- pom.xml
+-- CW_4
    +-- AiSD2026ZEx4
        +-- pom.xml
```

2.2 Zadanie 1

Proszę zapoznać się z kodem źródłowym dołączonym do ćwiczenia. Proszę zwrócić uwagę na metody interfejsu `MapInterface` implementowane w klasie `RbtMap`. Klasa implementująca mapę (`RbtMap`) wykorzystuje klasę `RedBlackTree` dla generycznych typów danych klucza (`K`) i wartości (`V`).

2.3 Zadanie 2

Analizę i zrozumienia kodu należy potwierdzić implementując testy (w przeważającej liczbie powinny to być testy o charakterze testów jednostkowych). Testy powinny weryfikować poprawność działania kodu zgodnie z ustaleniami podanymi w trakcie pierwszego spotkania dotyczącego bieżącego (czwartego) ćwiczenia.

Przy wykonaniu tego ćwiczenia proszę wspierać się analizą pokrycia kodu testami – aby to ułatwić, w konfiguracji projektu (`pom.xml`) została dołączona wtyczka z biblioteką **JaCoCo**.

Chcąc sprawdzić aktualne pokrycie kodu testami należy wykonać fazę **package** cyklu życia projektu maven-owego:

- z poziomu NetBeans-a: klikamy prawy przycisk myszy (PPM) na projekcie **-> Run Maven -> Goals** lub **Other Goals** zależnie od wersji IDE **->** w polu oznaczonym **Goals** wpisujemy: **package**;
- lub z wiersza poleceń: `mvn clean package`.

Po udanym zakończeniu fazy pakowania w głównym katalogu projektu powstanie katalog **target**, a w nim powinny być: **site -> jacoco -> index.html**. W ulubionej (lub tej dostępnej pod ręką) przeglądarce otwieramy **index.html** i analizujemy wynik. Staramy się, aby procentowe pokrycie kodu testami było jak największe.

2.4 Zadanie 3

W pakiecie testów (`test/java/pl/edu/pw/ee/aisd2026zex4/performance`) proszę stworzyć klasę `DataGenerator` z metodami generującymi dane (o rozmiarze `size`) w trzech porządkach (losowym, rosnącym, malejącym):

- `public String[] generateRandData(int size)`;
- `public String[] generateAscData(int size)`;
- `public String[] generateDescData(int size)`.

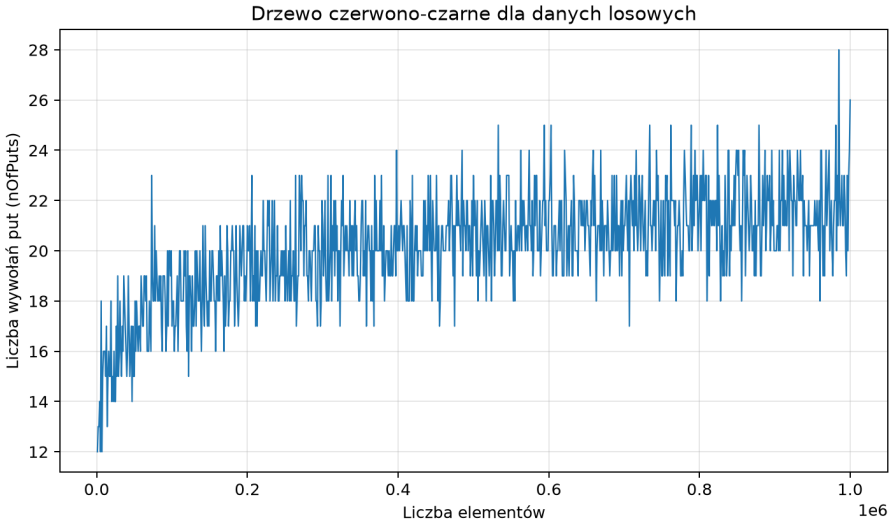
2.5 Zadanie 4

W klasie `RedBlackTree` jest prywatna metoda usuwająca „najmniejszy” klucz (wraz z wartością). Należy uzupełnić jej implementację na podstawie własnej intuicji lub korzystając z poniższego opisu dla usuwania klucza „największego” (a nie „najmniejszego”**!**).

0. Zabezpieczamy się przed usunięciem "czegoś" z pustego drzewa.
1. Sprawdzamy, czy lewy potomek naszego węzła jest czerwony.
   - FALSE: nic nie zmieniamy.
   - TRUE: wykonujemy rotację w prawo na naszym węźle i w ten sposób naszym nowym węzłem staje się wynik rotacji.
2. Sprawdzamy, czy przypadkiem nie dotarliśmy już do węzła, który nie ma prawego potomka.
   - FALSE: nic nie zmieniamy.
   - TRUE: zwracamy null jako potwierdzenie (w wywołaniu rekurencyjnym zwrócony „null" zostanie zapisany jako prawy potomek rodzica, który przed chwilą miał największego (w całym drzewie) prawego potomka).
3. Sprawdzamy, czy prawy potomek naszego węzła NIE jest czerwony (co nie oznacza, że jest czarny) oraz czy lewy potomek prawego potomka naszego węzła NIE jest czerwony (co też nie oznacza, że jest czarny).
   - FALSE: nic nie zmieniamy.
   - TRUE: (ani prawy, ani lewy od prawego nie jest czerwony) robimy zamianę kolorów.
   - TRUE.A. Sprawdzamy, czy nasz węzeł ma lewego potomka i czy lewy potomek tego lewego potomka jest czerwony.
     - TRUE.A.FALSE: nic nie robimy
     - TRUE.A.TRUE: wykonujemy rotację w prawo na naszym węźle i w ten sposób naszym nowym węzłem staje się wynik rotacji. Następnie jeszcze raz wykonujemy zamianę kolorów.
4. Robimy wywołanie rekurencyjne naszej metody usuwającej i jako parametr wywołania przekazujemy prawego potomka od naszego węzła. A wynik rekurencyjnego wywołania przypisujemy jako prawy potomek naszego węzła. (W ten sposób nawiązujemy do punkt 2.)
5. Wykonujemy naprawę naszego drzewa (tylko w tym węźle, w którym aktualnie jesteśmy).
   (a) Wykonujemy rotację w lewo (jeśli prawy potomek jest czerwony).
   (b) Wykonujemy rotację w prawo (jeśli obydwa kolejne lewe potomki są czerwone).
   (c) Wykonujemy zamianę kolorów (jeśli prawy potomek i lewy potomek są czerwone).
6. Jako wynik działania metody rekurencyjnej zwracamy wynikowy węzeł po operacjach z punktu 5.

2.6 Zadanie 5

Proszę, wykorzystując suplement z kodem źródłowym (załącznik z ISOD-a), przygotować wykres (rys. 1) pokazujący, jak zmienia się wysokość drzewa czerwono-czarnego (jednostronnie, lewostronnie czerwonego), gdy wstawiamy do niego klucze w porządku losowym. Proszę, odwołując się do teorii i swojej wiedzy, ustalić, czy wynikowy wykres jest „sensowny". Proszę odpowiedzieć, jak wyglądałby wynikowy wykres, gdyby wstawiane dane (klucze) były wstawiane w porządku rosnącym. A jak wyglądałyby dane wstawiane w porządku malejącym?



Rysunek 1: Liczba wywołań rekurencyjnej metody `put` podczas wstawiania danych losowych do drzewa czerwono-czarnego.

Wnioski:

Wykres wygląda sensownie: wraz ze wzrostem liczby elementów liczba wywołań `put` ogólnie rośnie, ale lokalnie może się zmniejszać. Wynika to z losowej kolejności kluczy i różnej głębokości, na której trafia kolejny element.

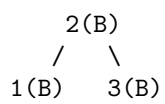
Dzięki rotacjom i zmianom kolorów drzewo pozostaje zrównoważone, więc wzrost jest zbliżony do logarytmicznego. Dla danych rosnących i malejących oczekiwany byłby podobny wynik. `nOfPuts` nie oznacza jednak dokładnej wysokości drzewa, lecz liczbę wywołań `put` dla ostatnio wstawianych danych.

2.7 Zadanie 6

Odwołując się do teorii (z wykładu lub materiałów uzupełniających: „Wprowadzenie do algorytmów", Cormen, Leiserson, Rivest, Stein), proszę podać przykład (sekwencję elementów wkładanych do (lub usuwanych z) początkowo pustego drzewa czerwono-czarnego) prowadzący do sytuacji wystąpienia węzła „podwójnie czarnego" w klasycznym RBT. Proszę krótko wyjaśnić, dlaczego dany przykład doprowadzi do takiej sytuacji.

Przykład:

Do początkowo pustego klasycznego drzewa czerwono-czarnego wstawiamy kolejno klucze $2, 1, 3$. Po zakończeniu wstawiania otrzymujemy:



Następnie usuwamy klucz 1. Jest to czarny liść, więc po jego usunięciu w jego miejscu pozostaje czarny węzeł wartowniczy NIL. Usunięcie czarnego węzła zmniejsza liczbę czarnych węzłów na ścieżce z korzenia, dlatego otrzymany węzeł NIL traktujemy jako „podwójnie czarny". Dodatkowa Czerń nie jest rzeczywistym kolorem zapamiętanym w węźle, lecz oznacza konieczność wykonania procedury naprawiającej własność czarnych wysokości drzewa.

2.8 Zadanie 7

Załóżmy, że do niepustego drzewa RB (jednostronnie, lewostronnie czerwonego) wstawiliśmy węzeł "x", a następnie (od razu po wstawieniu) usunęliśmy ten węzeł. Czy drzewo, które powstało po usunięciu "x" jest takie samo jak drzewo przed wstawieniem "x"?

Proszę podać uzasadnienie wraz z narysowaniem (właściwych z punktu widzenia odpowiedzi) sekwencji tworzenia drzew. Proszę oszczędnie wykorzystywać miejsce – odpowiedź powinna zmieścić się na jednej stronie.

Miejsce na odpowiedź i rysunki: TODO ——————.

2.9 Zadanie 8

Proszę zmodyfikować metodę testową `countTreeHeightDependingOnDataSize` w taki sposób, aby po zakończeniu pętli `while` była wyświetlana (wcześniej trzeba policzyć) liczba czerwonych węzłów wynikowego drzewa – proszę uzupełnić tabelę 1 zebranymi wynikami. Proszę skomentować uzyskane wyniki.

Być może, aby zrealizować to zadanie, trzeba będzie zmodyfikować kod w klasach: `RbtMap`, `RedBlackTree`, aby dostać się do korzenia drzewa. A czy da się to osiągnąć bez modyfikowania kodu? Wcześniej korzystaliśmy z mechanizmu refleksji. Czy tutaj również można go zastosować? Jeśli się Państwu uda, to proszę się tym pochwalić w dodatkowym komentarzu pod tabelą.

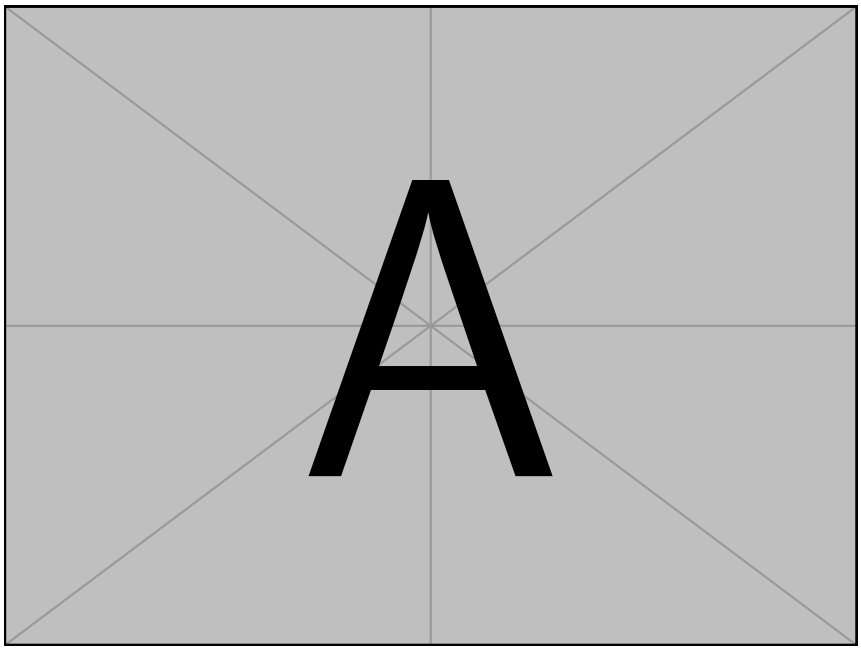
Rodzaj danych	Liczba czerwonych wezłów
Losowe	
Rosnące	
Malejące	

Tabela 1

Miejsce na komentarz do tabeli TODO ——————.

2.10 Zadanie 9

Proszę na rys. 2 zamieścić wynik (zrzut ekranu z głównej strony wyników JaCoCo) pokrycia kodu testami z najnowszej (ostatniej, uzupełnionej) wersji kodu.



Rysunek 2: Wynik pokrycia kodu testami (JaCoCo).

3 Na zakończenie

To już prawie koniec, ale tak jak poprzednio nie wolno nam zapomnieć, że:

- wszystkie zmiany (cały kod źródłowy) muszą trafić do repozytorium. A ostatnią zmianę (*commit*) w repozytorium zdalnym należy oznaczyć tagiem **ze znacznikiem czasowym CW_4_FINAL**;
- końcową wersję tego dokumentu (PDF) należy umieścić w ISOD-zie;
- **uwaga:** niewykonanie któregoś z powyższych dwóch kroków będzie oznaczało niewykonanie ćwiczenia, co jest równoznaczne z brakiem zaliczenia.